



nextwork.ai

Build a RAG API with FastAPI



Shivam Arora



Introducing Today's Project!

In this project, I'm going to build a RAG pipeline using FastAPI and ChromaDB. This will help me understand how RAG work How to implement it in the LLM. I'm interested in this because I love gaining knowledge about AI

Key tools and concepts

The key tools I used include FastAPI to build and run the local REST API server, ChromaDB as the persistent vector database to store and perform semantic searches on text documents, and Ollama to run both the nomic-embed-text embedding model and the lightweight qwen2.5:0.5b large language model entirely on my local machine. Key concepts I learnt include the fundamentals of the RAG (Retrieval-Augmented Generation) pipeline: how documents are broken into text chunks, how an embedding model maps semantic meaning into high-dimensional vector space coordinates, and how an LLM uses injected context to generate grounded, hallucination-free responses. Additionally, I learned about multi-tenancy patterns by using metadata filtering (where clauses) in ChromaDB to isolate specific users' data during queries.

Challenges and wins

Here is a polished answer you can use to fill in this final text box on your NextWork project page: This project took me approximately 45 to 60 minutes to complete. The most challenging part was understanding how the embedding model (nomic-embed-text) maps text into high-dimensional vector space coordinates and then figuring out how ChromaDB calculates the semantic distance between vectors to retrieve the correct context during a query. Building the multi-user extension with Pydantic schemas and using a where clause for metadata filtering was a fantastic way to see how real-world multi-tenant production RAG systems actually work!



Performing RAG Manually

In this step, I'm going to do a manual RAG demo to help you understand it. RAG stands for * Retrieval * Augmented * Generation

```
>>> Based on this information about me:
... "My name is Shivam Arora. I'm learning cloud computing and AI.
... My goal is to become a AI Engineer.
... I enjoy Gyming."
... What are my career goals?
...
Your career goal of becoming a "AI Engineer" by focusing on cloud computing and artificial intelligence aligns
well with your experience as a "Gyming" individual. Here's a more detailed breakdown of how you might achieve
this:

### Personal Goals:
1. **Increase Your Knowledge**: Continue to learn about AI, machine learning, and related technologies.
2. **Gain Industry Expertise**: Stay updated on the latest advancements in cloud computing and AI.
3. **Develop Skills**: Enhance your technical skills by contributing to projects related to AI and cloud systems.

### Career Goals:
1. **Acquire Advanced Skills**: Improve your expertise in both cloud computing (e.g., AWS, Azure) and AI (e.g., TensorFlow, PyTorch).
2. **Build a Strong Network**: Connect with professionals who can guide you and provide valuable insights into the industry.
3. **Leverage Your Existing Experience**: Build upon your current skills by leveraging them in projects that align with your career goals.

### Specific Areas to Consider:
1. **Cloud Computing**: Stay updated on new platforms, tools, and technologies like AWS, GCP, Azure, and Heroku.
2. **Machine Learning/Deep Learning**: Study deep learning frameworks and explore applications in areas like computer vision, natural language processing, and autonomous systems.
3. **AI Applications**: Explore various applications of AI within the healthcare, finance, transportation, and other sectors.

### How to Achieve Your Goals:
1. **Take Up Courses**: Online platforms and workshops can be a great way to learn new technologies related to cloud computing and AI.
2. **Join Communities**: Participate in forums, meetups, and social media groups related to AI and cloud computing.
3. **Build a Portfolio**: Create projects that showcase your skills, such as building an application for a project you are working on or solving a complex problem using AI.

### Practical Steps:
1. **Set Specific Goals**: Define what success means to you (e.g., learning new technologies, becoming proficient in specific areas).
2. **Plan Your Courses and Workshops**: Choose courses that align with your goals and interests.
3. **Build a Network**: Attend workshops, seminars, and conferences related to AI and cloud computing.
4. **Collaborate**: Engage with peers who have similar career aspirations.
5. **Seek Feedback**: Share your projects with experienced professionals in the field.
```

Understanding the three parts of RAG

I performed RAG manually by giving a prompt about my personal details. The three parts are Retrieval - find the context Augmentation - add the context Generation -use the context (AI does this part)



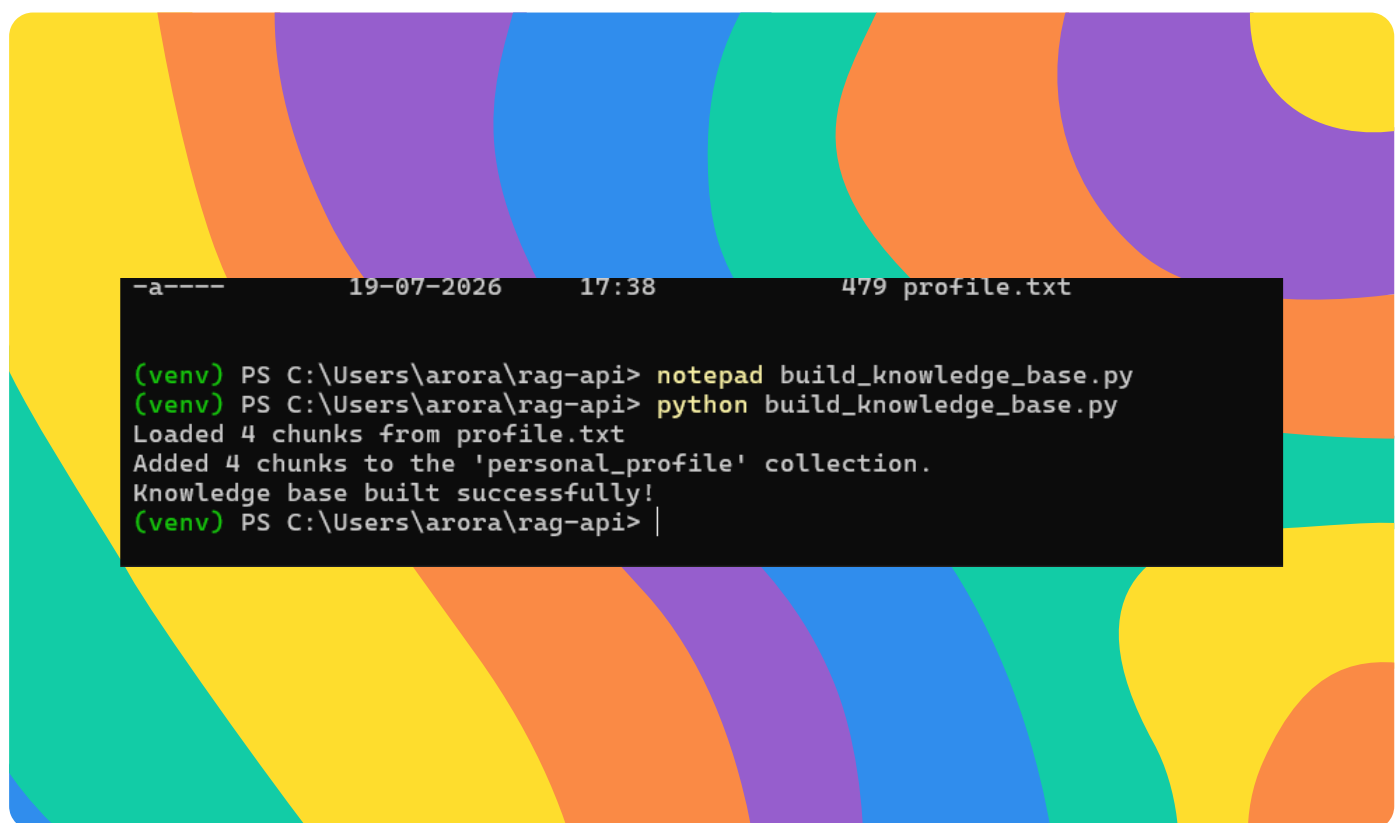
Comparing the two AI models

The key difference I noticed is `nomic-embed-text` is an Embedding Model: It doesn't chat, answer questions, or generate new text. Instead, its sole job is to take text and convert it into numerical representations called vectors. These vectors capture the mathematical meaning of the text, allowing databases like ChromaDB to perform semantic searches (finding text with similar meanings rather than just exact keyword matches). `qwen2.5:0.5b` is a Large Language Model (LLM): This is a generative, conversational model. It takes a text prompt (like a question combined with retrieved context) and generates a natural language response. The 0.5 b indicates it is a highly lightweight version with 500 million parameters, making it incredibly fast to run locally on your machine.



Building a Personal Knowledge Base

In this step, I'm going to create a personal profile, chunk the text, and store embeddings in Chroma DB. Embeddings are numerical representations of the meaning behind the text.



```
-a-----19-07-2026    17:38    479 profile.txt

(venv) PS C:\Users\arora\rag-api> notepad build_knowledge_base.py
(venv) PS C:\Users\arora\rag-api> python build_knowledge_base.py
Loaded 4 chunks from profile.txt
Added 4 chunks to the 'personal_profile' collection.
Knowledge base built successfully!
(venv) PS C:\Users\arora\rag-api> |
```

Creating the profile document

I included information about my background, my learning focus on Machine Learning, AI, and Data Science, my career goal of becoming an AI Engineer, and my hands-on project experience on NextWork. This text will be split into paragraphs (chunks) and converted into mathematical vector embeddings by ChromaDB. When I ask a personal question, the API will convert my question into a vector, locate the most semantically similar chunks from this profile, and feed them to the LLM as context so it can generate an accurate, grounded response.



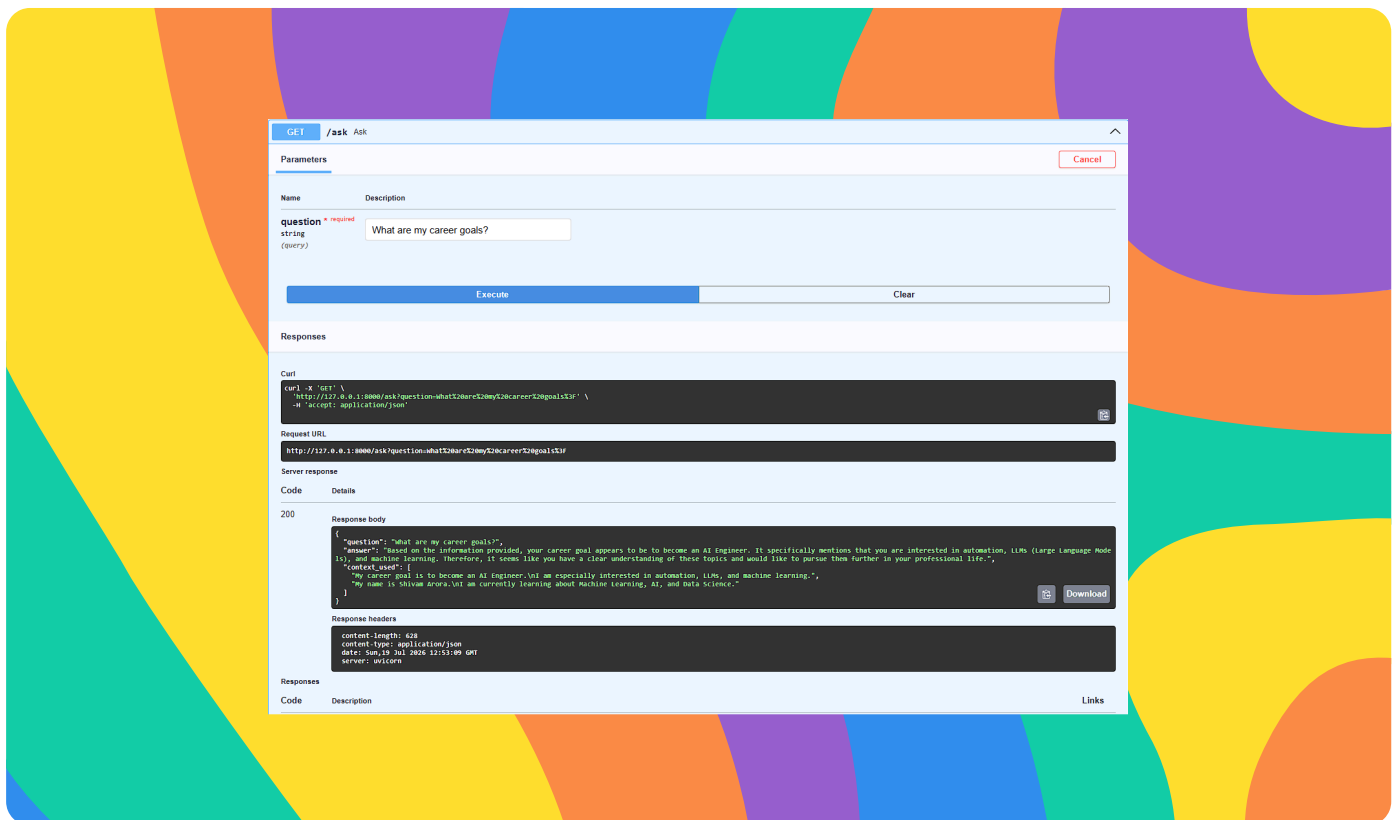
How semantic search finds relevant chunks

When I ask a question, ChromaDB converts the question into vectors and finds the chunks whose vectors are closest in the high-dimensional space. Semantic Search!



Creating the RAG API with FastAPI

In this step, I'm going to build an API that has an /ask endpoint. It's going to answer questions using grounded answers! I'll test it using Swagger UI.



How the /ask endpoint works

When a question comes in, my endpoint finds the most relevant chunks by querying ChromaDB, then passes those through the LLM to generate the grounded response.



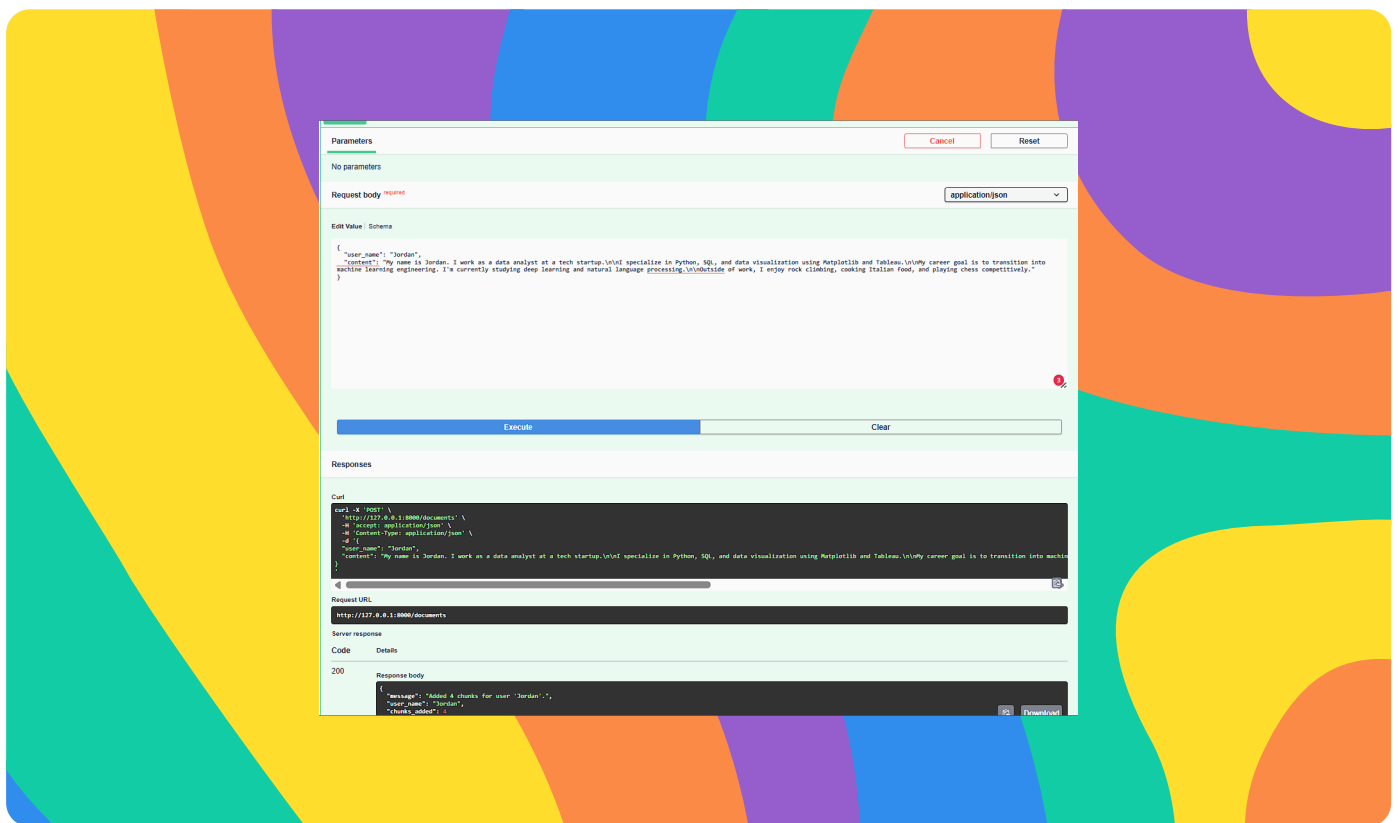
Testing with Swagger UI

I tested my API by asking, "What are my career goals?" The AI successfully answered with a grounded response stating that my career goal is to become an AI Engineer, with a specific focus on areas like automation, LLMs, and machine learning. The context used by ChromaDB consisted of the specific paragraphs retrieved from my profile.txt file that contained my background, tech stack, and career ambitions. By feeding this exact text into the qwen2.5:0.5b model alongside my question, the API ensured the AI's response was completely accurate and free of hallucinations.



Extending to a Multi-User AI Directory

In this project extension, I'm adding multi-user support because enterprise-level RAG systems rarely manage data for just a single user or resource. Multi-tenancy means the application serves multiple distinct users or teams while ensuring complete data isolation and privacy. Without multi-user support, a vector database treats all incoming documents as one shared pool of context. This leads to information leaking across accounts and causes the LLM to hallucinate by mashing unrelated documents together. Implementing metadata filtering fixes this issue by restricting semantic searches to a specific user's dataset.

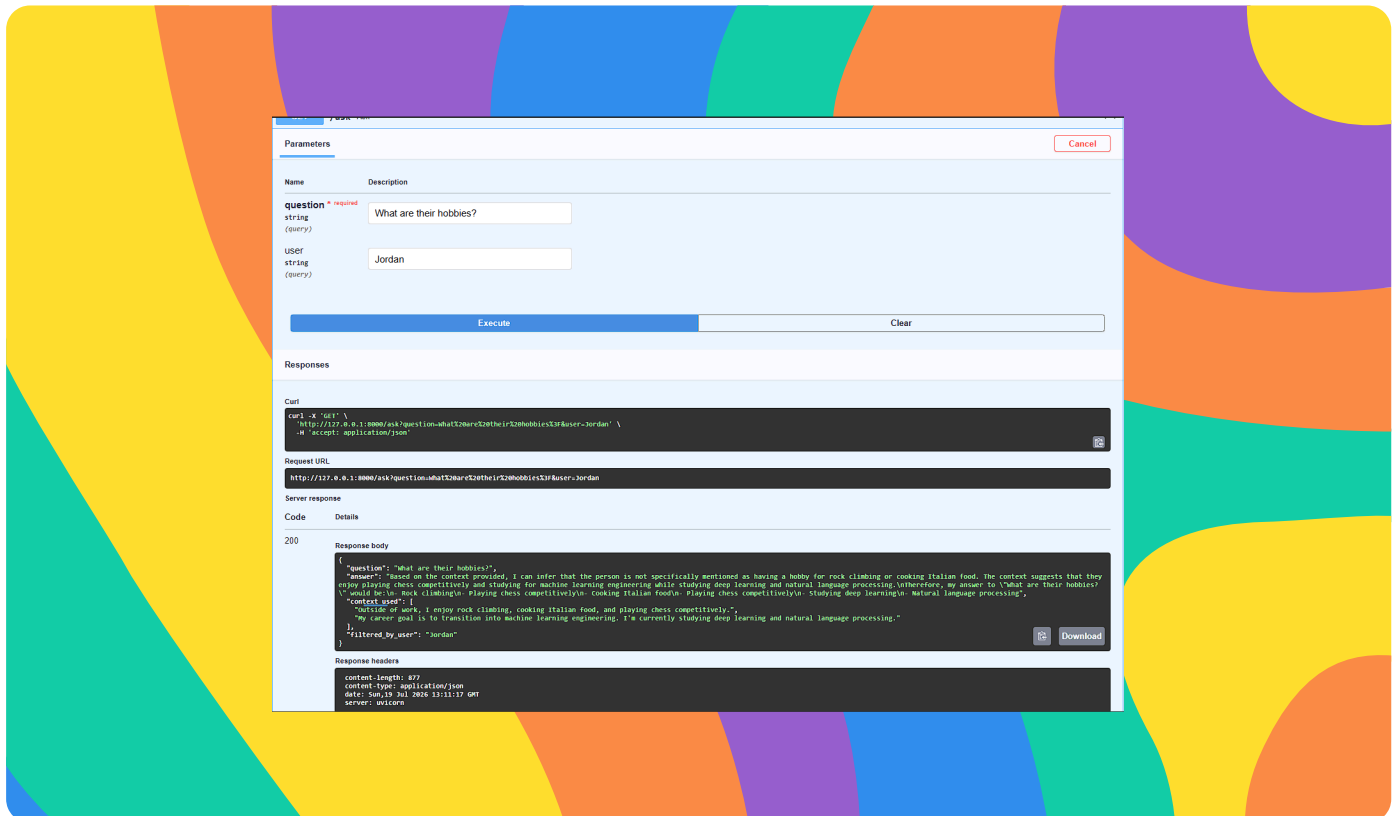


Adding the POST /documents endpoint

In this project extension, I added a POST endpoint that accepts a JSON payload containing a user_name and content. It splits the profile text into individual paragraph chunks and stores them in ChromaDB by attaching the provided user_name as a metadata dictionary key ({ "user_name": submission.user_name }) alongside each chunk. Metadata filtering allows us to



Instead of searching through every document in the collection, ChromaDB isolates and performs a semantic search only on the chunks belonging to that specific user, ensuring that context from different profiles doesn't get mashed together.



Verifying multi-user filtering

In this project extension, I tested multi-user queries by passing the user parameter to the /ask endpoint (e.g., setting user=Jordan) while asking a specific question like "What are their hobbies?". The filter works because passing a where clause targeting the specific user_name instructs ChromaDB to completely ignore chunks belonging to other profiles.



Shivam Arora

NextWork Student

nextwork.ai

I verified this by looking at the `context_used` field in the API's JSON response, which contained only the chunks from the intended user's text profile, and the AI's generated response accurately outlined only that specific user's interests without combining or leaking information from any other profile in the directory.



Wrapping Up

I did this project today to learn how to build a local Retrieval-Augmented Generation (RAG) API using FastAPI, ChromaDB, and Ollama. The most valuable part of the project was implementing the multi-user extension, which showed me how data isolation works through metadata filtering—a crucial concept for real-world applications. Another skill I want to learn next is containerizing this entire local pipeline using Docker to simplify deployment, followed by exploring advanced chunking strategies to handle complex, multi-page documents.



nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

